

# Shiv Nadar University Chennai

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |

## Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

### 1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

### 2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

### 3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

#### Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$  : Input vector
- $\mathbf{w}$  : Weight vector
- $b$  : Bias
- $z$  : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where  $\eta$  denotes the learning rate.

## 4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

### Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

| Activation | Output Range        | Typical Applications                    |
|------------|---------------------|-----------------------------------------|
| Step       | $\{0, 1\}$          | Classical Perceptron                    |
| Sigmoid    | $(0, 1)$            | Binary Classification Output Layer      |
| Tanh       | $(-1, 1)$           | Hidden Layers (Traditional Networks)    |
| ReLU       | $[0, \infty)$       | Hidden Layers in CNNs and MLPs          |
| Leaky ReLU | $(-\infty, \infty)$ | Avoiding Dying ReLU                     |
| Softmax    | $(0, 1)$            | Multi-class Classification Output Layer |

**Note:** This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

## 5. Dataset Description

**Dataset:** Banknote Authentication Dataset

**Source:** UCI Machine Learning Repository

**Download Link:**

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

|                |                       |
|----------------|-----------------------|
| Instances      | 1372                  |
| Features       | 4 Numerical Features  |
| Classes        | 2                     |
| Missing Values | None                  |
| Task           | Binary Classification |

### Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

## 6. Experimental Procedure

### Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

### Expected Output

Dataset summary and statistical information.

### Task 2: Exploratory Data Analysis

Generate

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

### **Task 3: Data Preprocessing**

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

### **Task 4: Perceptron Implementation**

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

### **Task 5: Model Training**

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

### **Task 6: Model Evaluation**

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

## 7. Source Code

The complete implementation of the Single Layer Perceptron was developed in Python using Jupyter Notebook. The implementation includes data preprocessing, exploratory data analysis (EDA), model training, performance evaluation, and visualization.

**GitHub Repository:**

[https://github.com/Mohula-Prasath/Deep\\_learning\\_lab/tree/main/lab-1](https://github.com/Mohula-Prasath/Deep_learning_lab/tree/main/lab-1)

## 8. Experimental Results

### 8.1 Dataset Exploration

The Banknote Authentication dataset was successfully loaded into the Jupyter Notebook. The dataset contains 1372 instances with four numerical input features and one binary target variable. No missing values were observed.

The following outputs were generated:

- First five samples of the dataset
- Dataset dimensions
- Dataset information
- Missing value analysis
- Descriptive statistics
- Target class distribution

### 8.2 Data Preprocessing

We used the Min-Max Scaler to make sure the numerical features were on the level. This means that all the features were scaled down to be between 0 and 1.

The dataset was then split into two parts: one for training and one for testing. We did this by taking 80 percent of the data for training. 20 Percent for testing. We made sure that the training and testing sets had the balance of classes by using a special method called stratified sampling, for the numerical features and the dataset.

- Feature Scaling: Min-Max Normalization
- Training Samples: 1097
- Testing Samples: 275
- Train-Test Ratio: 80:20

### 8.3 Model Training

The Single Layer Perceptron was made from the beginning using the Step Activation Function.

We started with all weights. The bias set to zero before we began training the Single Layer Perceptron. During each epoch of training the Single Layer Perceptron, the perceptron learning rule changed the weights and bias of the Single Layer Perceptron whenever it got a training sample wrong. The Single Layer Perceptron was trained for, up to 100 epochs using three learning rates: 0.001, 0.01 and 0.1. When we compared the learning rates of the Single Layer Perceptron we saw that all three learning rates had a trend when it came to convergence. So we kept the parameters of the Single Layer Perceptron that gave us the training error when we were training the Single Layer Perceptron.

The final model parameters are summarized below:

- Number of Epochs : 100
- Final Weight ( $w_1$ ) : -0.015552388212217485
- Final Weight ( $w_2$ ) : -0.01819735267374384
- Final Weight ( $w_3$ ) : -0.01774085317417528
- Final Weight ( $w_4$ ) : 0.00010315975101792289
- Final Bias : 0.022000000000000013
- Minimum Training Error : 16 Misclassified Samples

### 8.4 Model Evaluation

The trained perceptron model was evaluated on the testing dataset using standard classification metrics. The obtained performance metrics are listed below.

- Accuracy : 0.9890909090909091
- Precision : 0.9917355371900827
- Recall : 0.9836065573770492
- F1-score : 0.9876543209876543

## 9. Generated Plots with Interpretation

### 9.1 Histograms

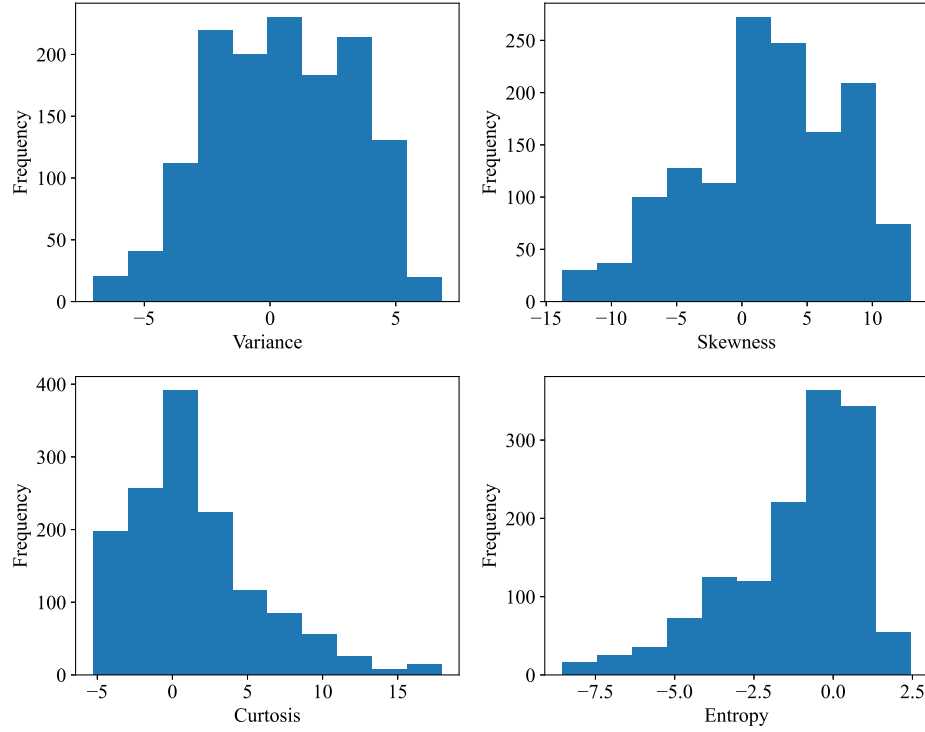


Figure 1: Histogram of Features

#### Interpretation:

All the histograms are mostly centric with either normally skewed or left skewed or right skewed. most of it does not contain any outlier except Feature 3 and Feature 4.

## 9.2 Correlation Heatmap

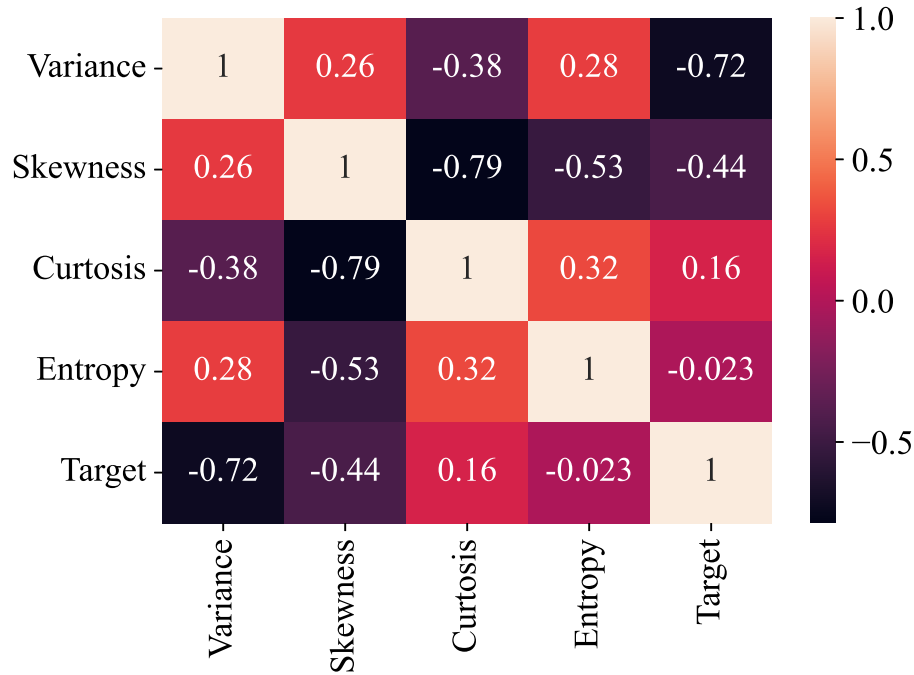


Figure 2: Heatmap

### Interpretation:

According to the correlation matrix Feature-1 is clearly most correlated to the Target variable with a negative correlation of -0.72. This makes Feature-1 the important predictor for the Perceptron. Feature-2 and Feature-3 have a correlation of -0.79, which means they are very similar and may compete with each other when training the Perceptron. In words these two features have a lot of redundancy. On the hand Feature-4 has almost no correlation with the Target variable at just -0.023. So Feature-1 is a factor and Feature-2 and Feature-3 may cause issues due to their high correlation. The Perceptrons performance will likely be influenced by how Feature-1 and the correlated features interact. Feature-1 and Target variable have a connection while Feature-4 does not. Feature-2 and Feature-3 should be examined closely due, to their correlation..

### 9.3 Scatter Plots

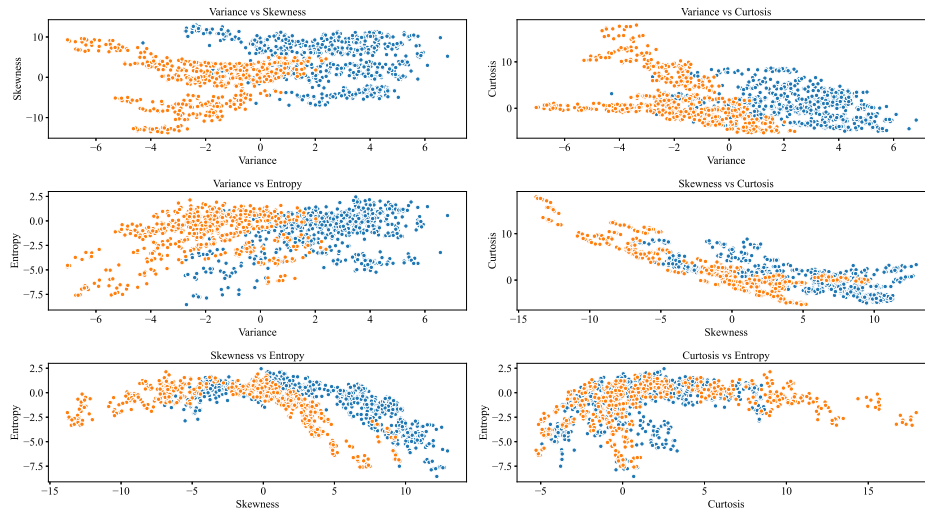


Figure 3: Scatterplot

#### Interpretation:

From the above scatter plots, the feature 1 and feature 2 almost linearly separates the target but not completely where there is a very high overlap between feature 3 and 4 where perceptron alone cannot distinguish target with just feature 3 and 4

## 9.4 Boxplots

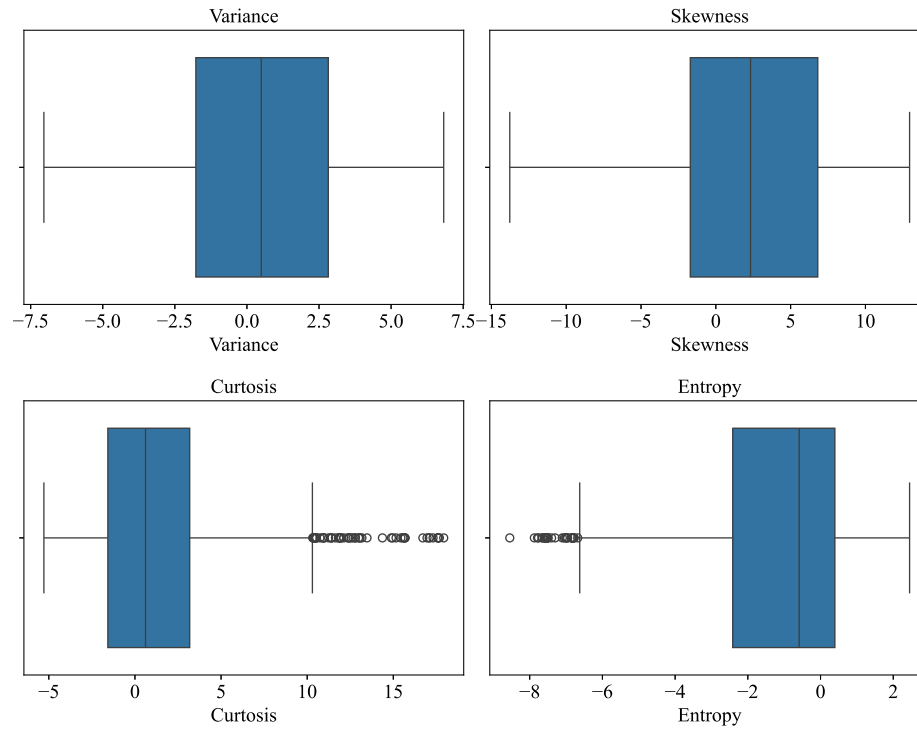


Figure 4: Boxplot

### Interpretation:

In the box plot, we notice a very symmetrically distributed data set for Feature 1 and 2 whereas the feature 3 is most left dominated having most of the outliers towards the 3rd quartile whereas feature 4 is right dominated and having most of its outliers on its left side.

## 9.5 Training Error vs Epoch

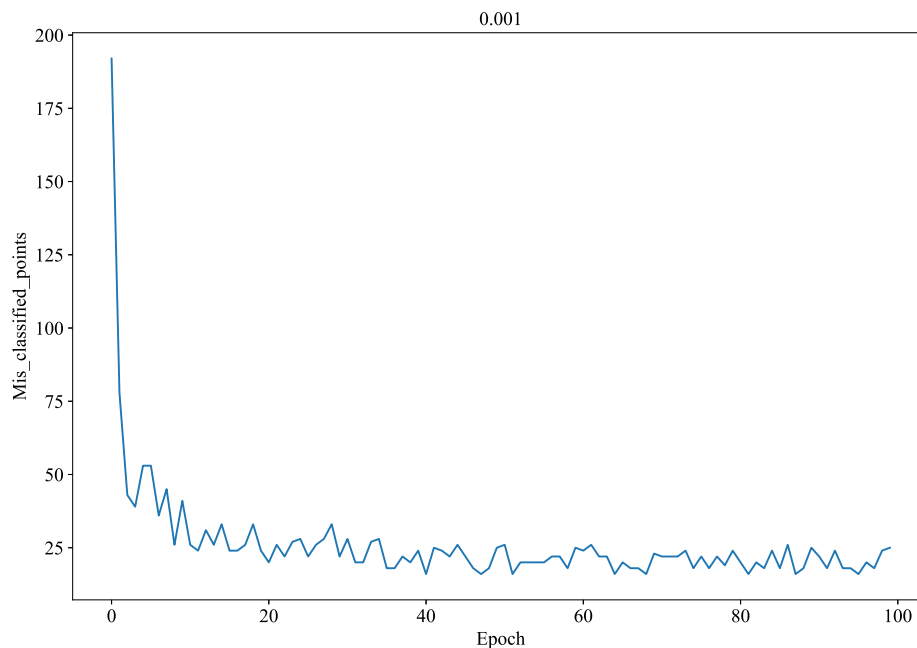


Figure 5: Training Error vs Epoch for lr=0.001

### Interpretation:

From the loss curve above, we can see that although the error rate decreases sharply at the beginning few epochs, it starts fluctuating between 15 and 100 and never touches zero. This confirms that your the points are not linearly separable.

## 9.6 Weight Evolution

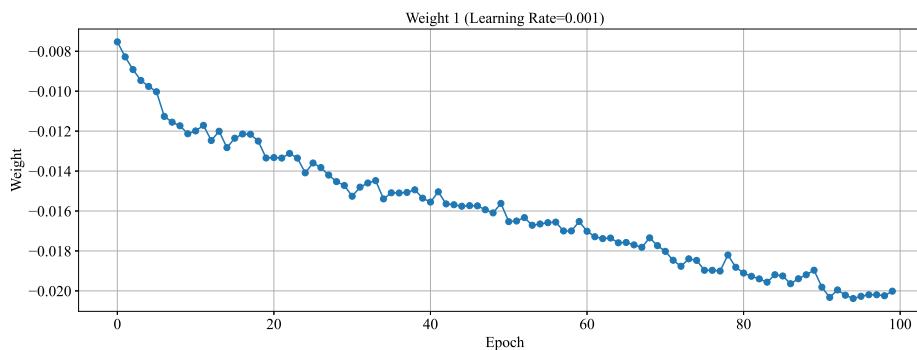


Figure 6: Weight progress for w1 with lr=0.001

### Interpretation:

The above weight progression graph shows a sharp decline during the throughout the 100 epochs but at last there was a very slight increase. the weight started from 0 and came all the way down to -0.02

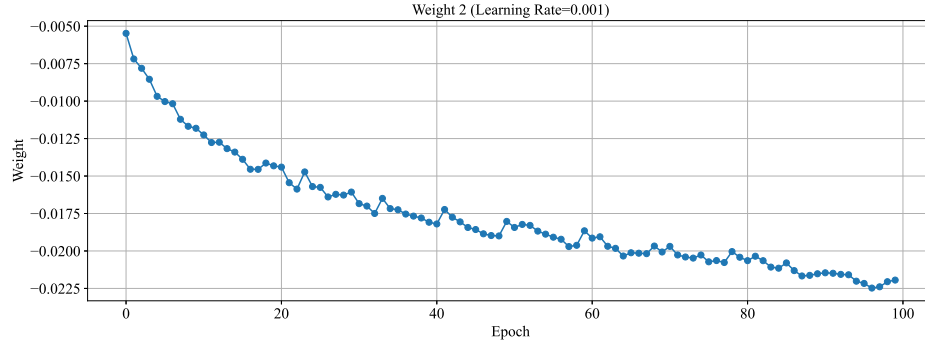


Figure 7: Weight progress for w2 with lr=0.001

**Interpretation:**

The weight starts declining sharply Just as previously seen. since there is no linear decision boundary that can be made, the weights keep fluctuating from one side to another while adjusting.

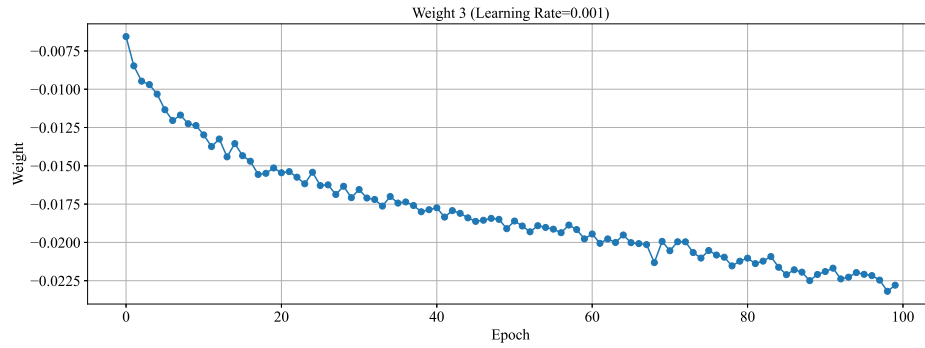


Figure 8: Weight progress for w3 with lr=0.001

**Interpretation:**

In this case, the graph steadily decreases which starts form 0 and came upto -0.025. From the graph we can see that the slope of the curve keeps decreasing as the epoch increases.

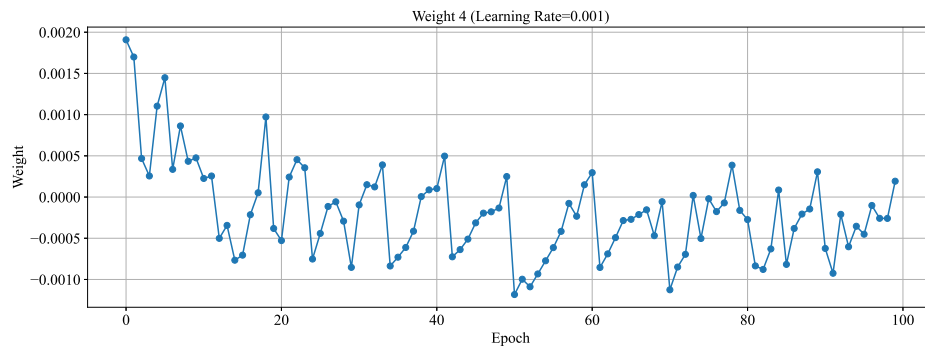


Figure 9: Weight progress for w4 with lr=0.001

**Interpretation:**

Differently from Weights 1, 2, and 3 that show gradual convergence towards steady negative values, the fourth weight demonstrates ongoing oscillations near zero during training. It shows that the value of this particular feature has little impact on the classification process and changes its sign frequently. Indeed, the feature in question demonstrates low correlation with the output variable, hence low importance to the decision boundary equation.

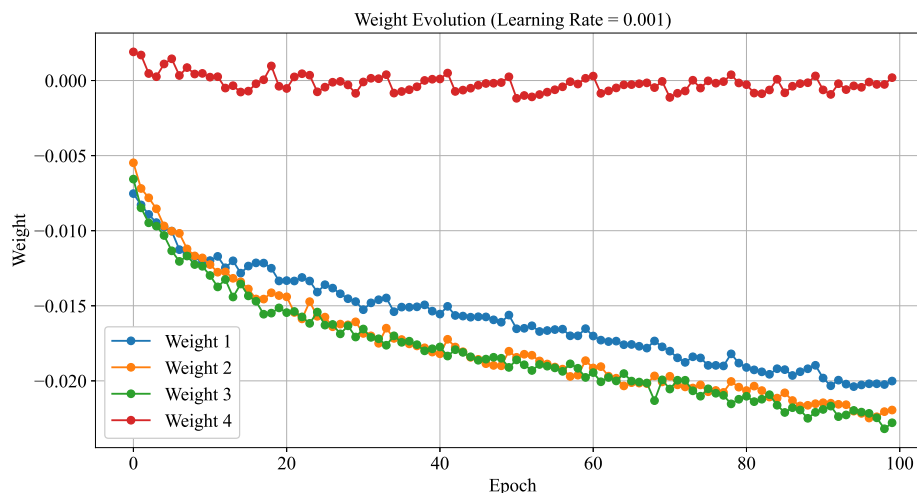


Figure 10: combined Weight progress for lr=0.001

### Interpretation:

The weight 1,2,3 almost starts at the same point and decreases gradually all the way to the 0.015 whereas feature 4 does not show any increase or decrease but fluctuates and ends at 0 itself

## 9.7 Bias Evolution

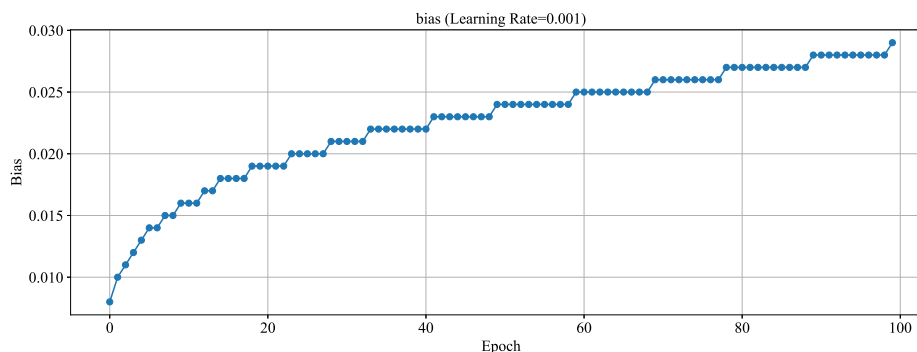


Figure 11: Bias progress with lr=0.001

### Interpretation:

From the graph, it can be seen that there is an increase in bias in the training process for learning rates of 0.001. This slow, stepwise change shows that there is learning happening, and the plateau areas show that there is no need to change the bias anymore. There is smooth convergence in the bias.

## 9.8 Confusion Matrix

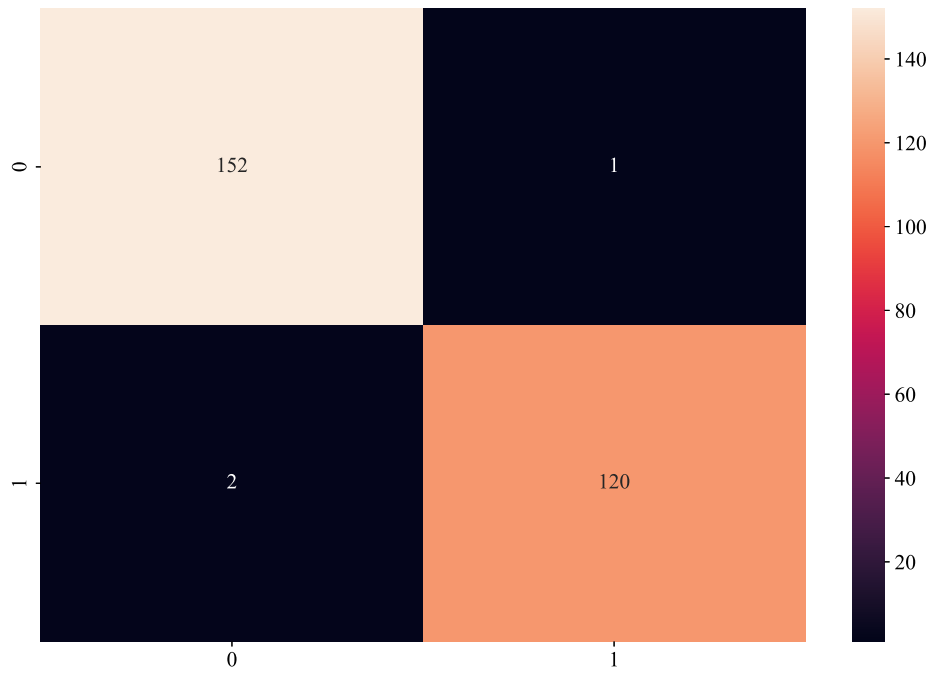


Figure 12: Confusion matrix

### Interpretation:

From the confusion matrix, it is clear that the perceptron classified 152 authentic banknotes correctly along with 120 forged banknotes. It only misclassified 3 instances out of the total number of samples (1 instance being a false positive and 2 instances being false negatives).

## 9.9 Learning Rate Comparison

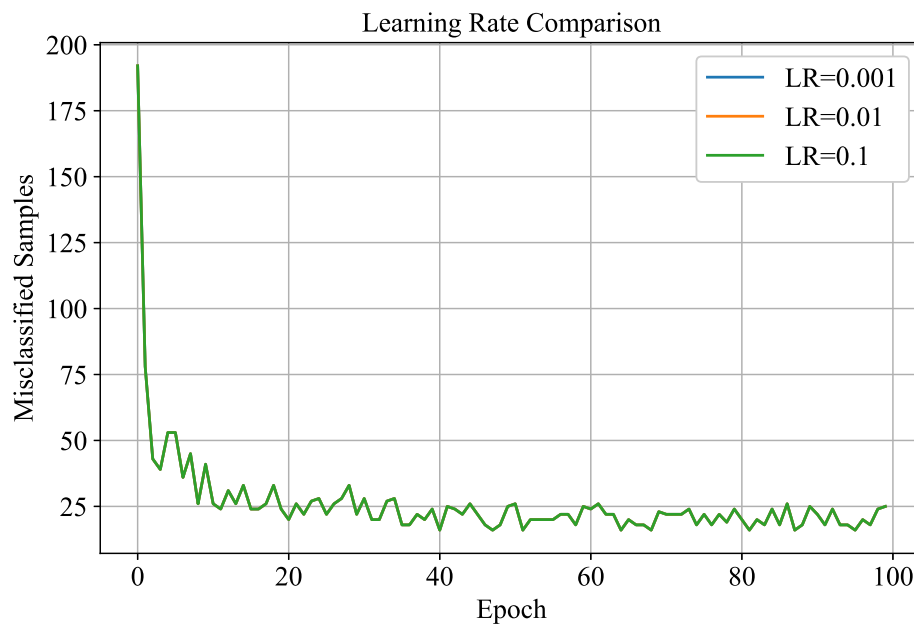


Figure 13: Learning Rate Comparison

**Interpretation:** The figure shows a comparison between the number of errors made while classifying the samples through epochs at various learning rates. The three learning rates display a similar trend where all the curves merged together to a single curve towards convergence.

## 10. Performance Tables

### 10.1 Training Summary

| Parameter        | Value                                                                                                                           |
|------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Dataset Size     | 1372 Samples                                                                                                                    |
| Train/Test Split | 1097 / 275 (80:20)                                                                                                              |
| Learning Rate    | 0.001                                                                                                                           |
| Epochs           | 100                                                                                                                             |
| Final Weights    | $w_1 = -0.015552388212217485$<br>$w_2 = -0.01819735267374384$<br>$w_3 = -0.01774085317417528$<br>$w_4 = 0.00010315975101792289$ |
| Final Bias       | 0.0220000000000000013                                                                                                           |
| Accuracy         | 0.9890909090909091                                                                                                              |
| Precision        | 0.9917355371900827                                                                                                              |
| Recall           | 0.9836065573770492                                                                                                              |
| F1-score         | 0.9876543209876543                                                                                                              |

### 10.2 Epoch-wise Learning

| Epoch | Errors | Weight 1 | Weight 2 | Bias  |
|-------|--------|----------|----------|-------|
| 1     | 192    | -0.00753 | -0.00548 | 0.008 |
| 2     | 78     | -0.00828 | -0.00719 | 0.010 |
| 3     | 43     | -0.00892 | -0.00781 | 0.011 |
| 4     | 39     | -0.00946 | -0.00854 | 0.012 |
| 5     | 53     | -0.00976 | -0.00968 | 0.013 |
| 10    | 41     | -0.01213 | -0.01181 | 0.016 |
| 20    | 24     | -0.01335 | -0.01432 | 0.019 |
| 30    | 22     | -0.01472 | -0.01607 | 0.021 |
| 40    | 24     | -0.01536 | -0.01808 | 0.022 |
| 50    | 25     | -0.01562 | -0.01803 | 0.024 |
| 80    | 24     | -0.01882 | -0.02042 | 0.027 |
| 90    | 25     | -0.01896 | -0.02152 | 0.028 |
| 100   | 25     | -0.02001 | -0.02194 | 0.029 |

## 11. Additional Tasks

### 11.1 Comparison of Step and Sigmoid Activation Functions

| Property                | Step Activation | Sigmoid Activation                                              |
|-------------------------|-----------------|-----------------------------------------------------------------|
| Output Range            | $\{0, 1\}$      | $(0, 1)$                                                        |
| Nature                  | Binary output   | Continuous probability output                                   |
| Differentiable          | No              | Yes                                                             |
| Gradient-based Learning | Not Possible    | Possible                                                        |
| Modern Deep Learning    | Not Used        | Widely Used (especially output layer for binary classification) |

#### Discussion:

The Step Activation Function can give only binary results (either 0 or 1). It is not a differentiable function. Modern-day deep learning models learn by means of backpropagation and gradient descent, and, therefore, they require gradient calculations for the updating of weight. In the absence of any derivative for the Step Activation Function, it is impossible to use it in gradient descent-based learning algorithms..

### 11.2 Comparison with Scikit-learn Perceptron

The implemented Single Layer Perceptron was compared with the Scikit-learn implementation using the same training and testing datasets.

| Model                   | Accuracy |
|-------------------------|----------|
| Implemented Perceptron  | 0.9891   |
| Scikit-learn Perceptron | 0.9818   |

The manually implemented perceptron achieved slightly higher accuracy than the Scikit-learn implementation. Both models produced similar results, showing that the implemented learning algorithm functions correctly.

### 11.3 Effect of Different Learning Rates

Three learning rates were used for the perceptron, namely 0.001, 0.01 and 0.01. It can be observed that the curve for all the three learning rates was converging in a similar manner. For all the three learning rates, there was a rapid decrease in the number of misclassification at the beginning of the epochs and finally it became stable at a relatively small number. As the data is not fully linearly separable, hence none of the learning rates was able to reach the zero error.

### 11.4 Why a Single Layer Perceptron Cannot Solve the XOR Problem

The XOR problem cannot be linearly classified because there is no line that can distinguish between the two classes of the XOR problem. The Single Layer Perceptron can classify using only linear

classification boundaries and, thus, cannot solve the XOR problem. The XOR problem needs to be solved using a Multi-Layer Perceptron with one or more hidden layers.

### **11.5 Effect of Feature Normalization on Model Convergence**

Feature normalization involves bringing all input features to the same range, hence avoiding dominance by those features whose values are high when computing their weights. This is important for stable learning, faster convergence and training. In this experiment, Min-Max feature normalization was done before training the perceptron, and this enabled the algorithm to consistently converge and score a very high accuracy of 98.91%.

### **11.6: Implementation of Perceptron Learning Algorithm for Logic Gates**

#### **Objective**

To implement the Perceptron Learning Algorithm for the OR, AND, and NOT gates using Python, keep track of the weight changes in each iteration and trace the path of the decision boundary during training.

#### **Experimental Procedure**

1. Create datasets for the OR, AND, and NOT logic gates.
2. Initialize all weights and bias to zero.
3. Apply the Perceptron Learning Algorithm using a step activation function.
4. Update the weights whenever a sample is misclassified.
5. Display the updated weights after every weight update.
6. Plot the decision boundary after each training epoch.
7. Repeat the process until all samples are correctly classified or the maximum number of epochs is reached.

#### **Experimental Results**

The perceptron could learn decision boundaries for the OR, AND and NOT logic gates. In training, the weights and bias were updated each time a misclassification was made. As the epochs passed, the number of misclassified samples decreased and eventually converged to zero, indicating that the learning algorithm converged.

## Plots and Interpretation

### OR Gate

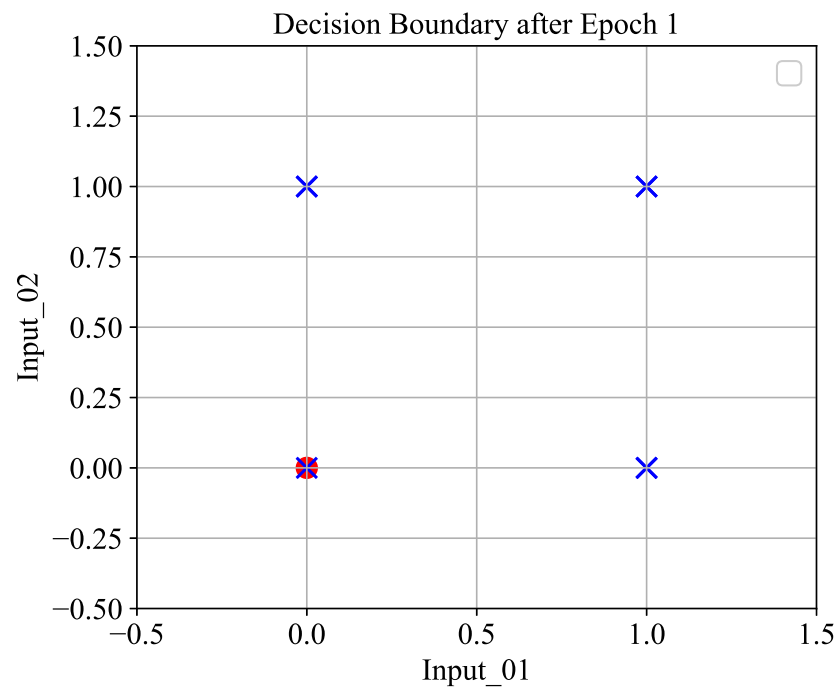


Figure 14: OR Gate Decision Boundary after Epoch 1

**Interpretation:** Since the weights and bias are being 0 at initial there was no decision boundary to classify all the samples

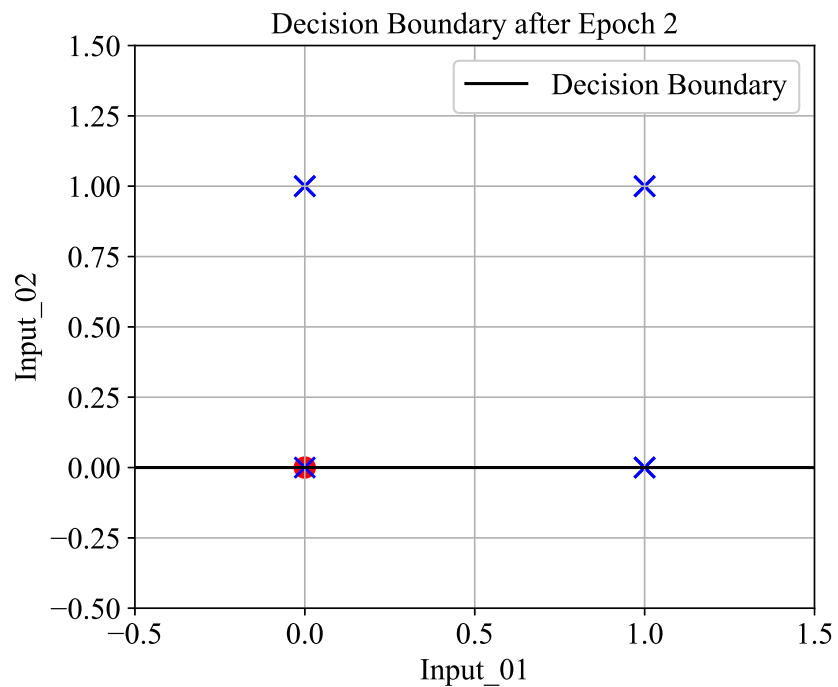


Figure 15: OR Gate Decision Boundary after Epoch 2

**Interpretation:** The first decision boundary cannot separate all the samples correctly so additional epoch is required

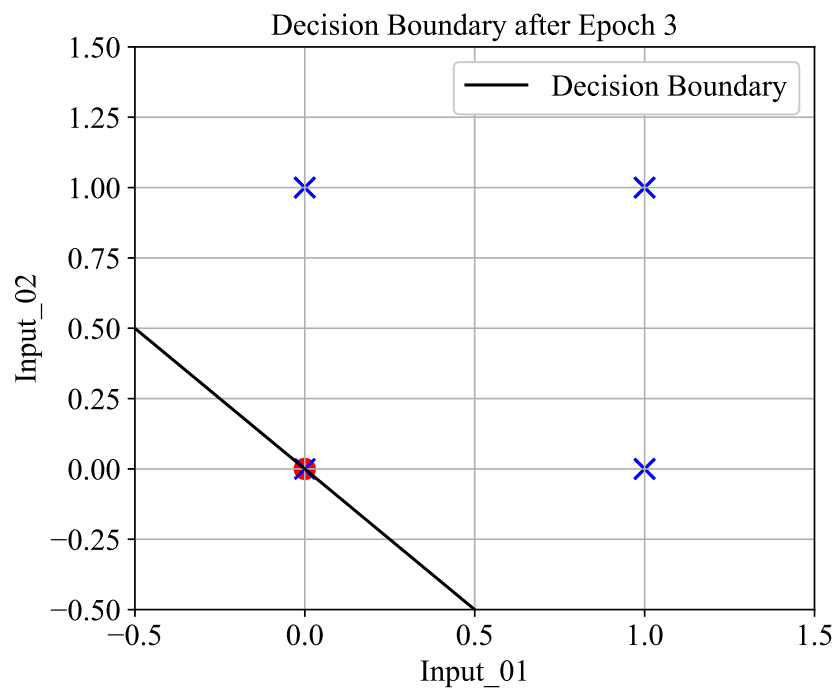


Figure 16: Final Decision Boundary for OR Gate

**Interpretation:** The perceptron converges successfully and the final decision boundary correctly separates the OR gate outputs. No further weight updates are required after convergence.

### AND Gate

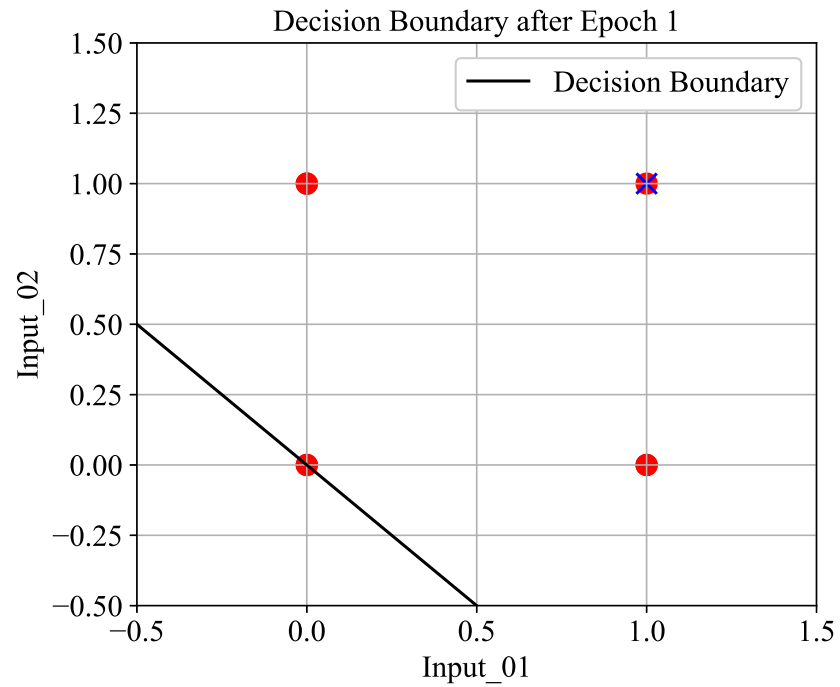


Figure 17: AND Gate Decision Boundary after Epoch 1

**Interpretation:** The first decision boundary is obtained by the perceptron updating its weights on the misclassified training samples. The boundary has begun to separate the data but some samples are still misclassified indicating that more weight updates are needed.

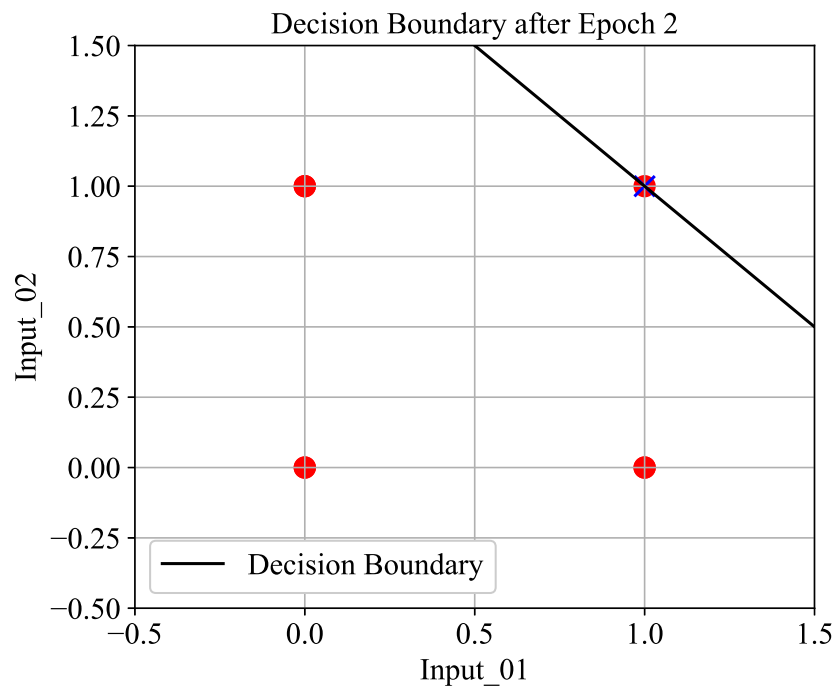


Figure 18: AND Gate Decision Boundary after Epoch 2

**Interpretation:** After the second training epoch, further weight updates move the decision boundary closer to the desired separating line. The number of misclassified samples was reduced, meaning that the perceptron learns slowly the decision function of the AND gate

## NOT Gate

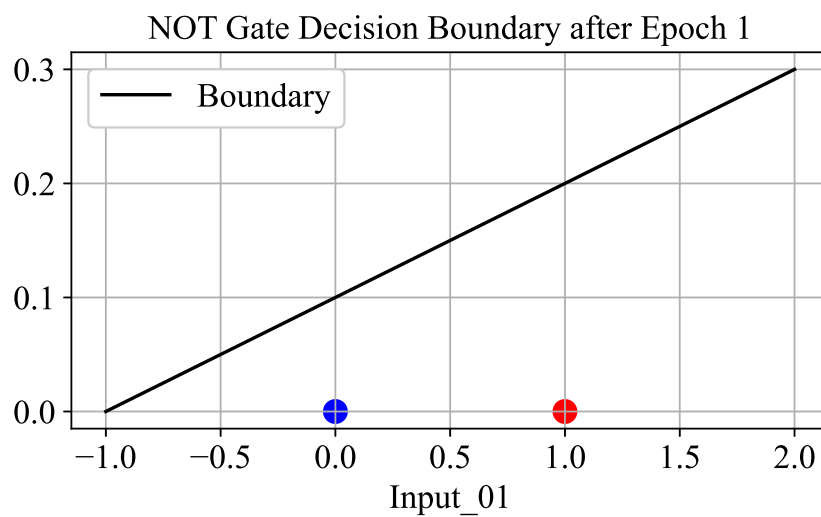


Figure 19: NOT Gate Decision Boundary after Epoch 1

**Interpretation:** During the first epoch, the perceptron performs its initial weight update based on the classification error. The decision boundary begins to shift toward the correct position, but the model has not yet completely learned the NOT gate logic.

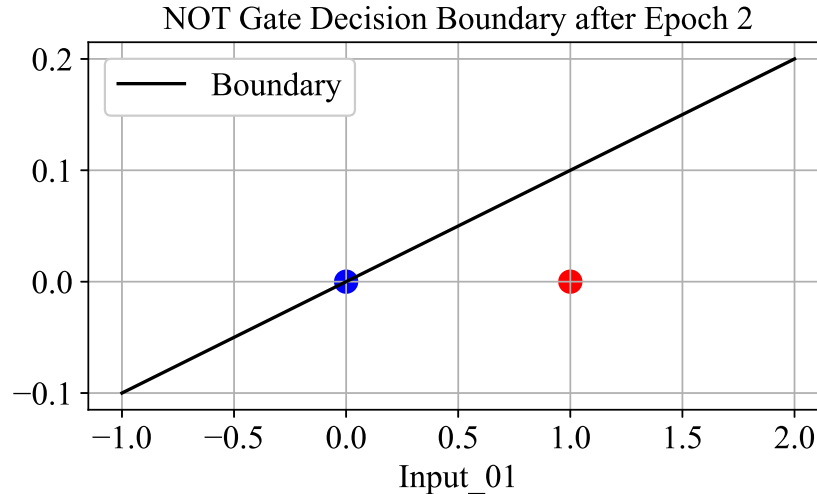


Figure 20: Final Decision Boundary for NOT Gate

**Interpretation:** After the end of the second epoch, the decision boundary moves to the right and correctly classifies the two classes of the NOT gate. As all the training data points are classified correctly, perceptron training has been achieved and weight updating is no longer needed.

## 12. Conclusion

The Single Layer Perceptron was implemented from scratch and tested on the Banknote Authentication dataset. After normalization of features and training with 100 epochs and a learning rate of 0.001, the network had an accuracy of 98.91%, a precision of 99.17%, recall of 98.36%, and an F1-score of 98.77%. Even though there was no convergence of training errors to zero because of overlapping data, the perceptron learned an efficient linear separating hyperplane that performed well on the test dataset. This experiment helped gain insight into the working of the perceptron learning algorithm and the effect of feature processing on the performance of classification.

## 13. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.